CineSuggest — Master Project Documentation

Document Type: Master Project Context Document (Single Source of Truth) **Primary Audience:** AI coding agents (Claude Code, Antigravity, Codex, and similar), human engineers **Document Status:** Living document — supersedes prior informal notes unless a newer approved document explicitly overrides it

> **Note on interpretation:** Wherever a decision has not been finalized by the project owner, this document explicitly writes **"Decision Pending"**, followed by a recommended option, alternative options, and the reasoning behind the recommendation. AI agents must not silently finalize these decisions — they should implement the recommended option only after confirming it with the project owner, or clearly flag the assumption if asked to proceed unattended.

Table of Contents

1. Project Overview
2. Problem Statement
3. Project Objectives
4. Scope
5. Target Users
6. Core Features
7. Cine Digest
8. User Roles
9. User Stories
10. Functional Requirements
11. Non-Functional Requirements
12. Technology Stack
13. Frontend Architecture
14. Page-Level Requirements

15. Dynamic Movie Card Architecture
16. Backend Architecture
17. Spring Boot Package Structure
18. Database Architecture
19. Detailed Table Schemas
20. Database Relationships
21. ER Diagram
22. Dataset Architecture
23. TMDb Data Strategy
24. MovieLens Data Strategy
25. Movie ID Mapping Strategy
26. Data Import Strategy
27. Rating System
28. Review System
29. Trailer System
30. Authentication and Security
31. REST API Architecture
32. Detailed API Specification
33. Recommendation Engine
34. Content-Based Filtering
35. TF-IDF Vectorization
36. Cosine Similarity
37. Item-Based Collaborative Filtering
38. Hybrid Recommendation System
39. Cold Start Strategy
40. Recommendation Execution Flow

41. Cine Digest Architecture
42. External API Integration
43. Complete System Architecture
44. Data Flow Diagrams
45. Detailed Feature Data Flows
46. Validation Rules
47. Error Handling
48. Performance Considerations
49. Testing Strategy
50. Development Workflow
51. Deployment Considerations
52. AI Agent Development Rules
53. Current Mandatory Scope
54. Future Enhancements
55. Glossary
56. Project Status at Documentation Creation
57. AI Context Quick Start

---

1. Project Overview

CineSuggest is a full-stack movie review, rating, cinema discovery, and personalized movie recommendation platform.

Users can create an account, browse a catalogue of thousands of movies, view rich movie detail pages (poster, overview, genres, release information, average rating), rate movies on a 1.0–5.0 scale with 0.5 increments, write and read reviews, watch official trailers, and receive personalized recommendations driven by a hybrid recommendation engine. A dedicated module called **Cine Digest** surfaces broader cinema trends (trending/popular/upcoming movies, new trailers, cinema news) independent of any single user's personal taste profile.

CineSuggest is explicitly **not** a hardcoded movie showcase. The movie catalogue is data-driven end-to-end: MySQL stores the data, Spring Boot exposes it through REST APIs, and vanilla JavaScript renders it dynamically in the browser using one reusable card template.

2. Problem Statement

Movie audiences are confronted with an overwhelming number of streaming and cinema options and typically rely on fragmented sources — separate apps for ratings, separate sites for trailers, separate feeds for "what's trending," and generic, one-size-fits-all recommendation feeds that don't meaningfully learn from the user's own rating behavior.

CineSuggest addresses this by unifying three distinct concerns behind one clean, dynamically driven interface:

Problem	CineSuggest's Answer
No single place to rate/review movies and see others' opinions	Ratings + Reviews modules backed by MySQL
Recommendations are generic or absent	Hybrid recommendation engine (content-based + item-based collaborative filtering)
Cinema trends and personal taste are conflated in most apps	Cine Digest is architecturally and conceptually separate from Personalized Recommendations
Large catalogues force either poor UX or unmaintainable hardcoded pages	Dynamic, API-driven movie card rendering from a single reusable template

3. Project Objectives

1. Build a working full-stack movie platform using Java/Spring Boot, MySQL, and vanilla HTML/CSS/JS — no unapproved framework substitutions.
2. Populate a large movie catalogue (thousands of movies) using TMDb metadata and MovieLens historical ratings, without manual data entry.
3. Implement a genuine hybrid recommendation engine (content-based + item-based collaborative filtering) rather than a superficial "trending movies" substitute.
4. Cleanly separate personalized recommendations from cinema-wide trend discovery (Cine Digest).
5. Implement secure authentication with hashed passwords and properly scoped authorization (users can only modify their own ratings/reviews).

6. Keep the frontend dynamic and reusable — one movie card template drives an unbounded catalogue.
7. Produce a documentation base detailed enough that AI coding agents can implement the system consistently, without re-deriving architecture decisions from scratch or silently substituting technologies.

4. Scope

In scope (current mandatory implementation):

- User signup/login/logout
- Movie browsing, searching, and detail viewing
- Star ratings (1.0–5.0, 0.5 increments) with create/update semantics
- Text reviews (create/read; update/delete for the review's own author)
- Trailer playback via external video reference (e.g., YouTube)
- Personalized recommendations via content-based filtering, TF-IDF, cosine similarity, item-based collaborative filtering, and a hybrid combination of the two
- Cine Digest (trending/popular/upcoming cinema content), architecturally separate from personalized recommendations
- MySQL-backed persistence with a normalized core schema (users, movies, ratings, reviews)
- REST API layer built with Spring Boot

Out of scope for the current mandatory implementation (see Future Enhancements):

- Watchlists, favorites, social following, review likes
- Sentiment analysis / advanced NLP
- Admin moderation dashboards
- React frontend migration, dedicated Python ML microservice, Redis caching, cloud deployment automation

5. Target Users

User Type	Description

Anonymous visitor	Can browse the catalogue, view movie details, view Cine Digest, and watch trailers, but cannot rate, review, or receive personalized recommendations
Registered user	Full access: rating, reviewing, personalized recommendations, profile
(Future) Administrator	Content moderation and catalogue management — **not** part of current mandatory scope

6. Core Features

Feature	Summary
Movie Catalogue	Thousands of movies, browsable and searchable, rendered dynamically from REST API data
Movie Details Page	Poster, title, overview, genres, release info, average rating, trailer, reviews
Rating System	1.0–5.0 in 0.5 increments; one active rating per user per movie; updatable
Review System	Free-text reviews, independent of ratings
Trailer Playback	Official trailer via external video reference (e.g. YouTube key/URL)
Personalized Recommendations	Hybrid engine combining content-based and item-based collaborative filtering
Cine Digest	Trending/popular/upcoming/new-trailer cinema content, distinct from personalized recommendations
Authentication	Signup/login/logout with hashed passwords and token-based session handling

7. Cine Digest

Cine Digest is a **dedicated page/module** inside CineSuggest that answers "what's happening in cinema right now" — independent of any individual user's personal taste profile.

Cine Digest may surface:

- Trending movies

- Popular movies
- Upcoming releases / now-playing movies
- New trailers
- Cinema news, awards updates, major announcements
- Popular actors/filmmakers when relevant to a trend

Cine Digest is explicitly NOT:

- The personalized recommendation engine
- Based on the logged-in user's rating history
- A place where content-based/collaborative filtering scores are computed

Aspect	Movie Catalogue	Personalized Recommendations	Cine Digest
Basis	Full stored dataset	Individual user's ratings + item similarity	Global cinema trends
Personalized?	No	Yes	No
Data Source	MySQL (TMDb + MovieLens derived)	MySQL ratings + precomputed similarity data	External cinema/movie API (e.g., TMDb trending endpoints)
Requires Login?	No	Yes	No
Updates	On catalogue import	On new ratings / scheduled recompute	On a refresh/cache interval

Architecturally, Cine Digest content flows from an **external data source → Spring Boot integration service → transformation → dedicated REST endpoint → frontend**, keeping it decoupled from the recommendation module. See Section 41 for full detail.

8. User Roles

Role	Capabilities

Anonymous	Browse catalogue, view movie details, view Cine Digest, watch trailers, read reviews
Authenticated User	All anonymous capabilities + rate movies, write/edit/delete own reviews, view personalized recommendations, manage own profile
(Future) Administrator	Moderate reviews, manage catalogue — not implemented in current mandatory scope

9. User Stories

ID	As a...	I want to...	So that...
US-01	Anonymous visitor	browse movies without an account	I can explore before committing to signup
US-02	Anonymous visitor	search for a specific movie	I can quickly find what I'm looking for
US-03	New user	sign up with email/username/password	I can create a personal account
US-04	Registered user	log in and stay authenticated	I can rate and review without re-entering credentials constantly
US-05	Registered user	rate a movie from 1.0–5.0 in 0.5 steps	I can express nuanced opinions
US-06	Registered user	update a rating I already gave	I can correct or change my opinion over time
US-07	Registered user	write a review	I can share detailed thoughts beyond a star rating
US-08	Any visitor	read other users' reviews	I can get more context before watching a movie
US-09	Any visitor	watch the official trailer	I can preview the movie
US-10	Registered user	receive personalized recommendations	I discover movies aligned with my taste
US-	Any visitor	view Cine Digest	I stay aware of what's trending in cinema, independent of my

11			personal history
US-12	New user with no rating history	still get sensible recommendations	I'm not shown an empty or broken screen (cold start)

10. Functional Requirements

1. The system shall allow account creation with a unique email and username.
2. The system shall hash passwords before persistence; plain-text storage is prohibited.
3. The system shall allow authenticated users to submit a rating between 1.0 and 5.0 in 0.5 increments.
4. The system shall enforce **one active rating per (user, movie) pair**; a repeat submission updates the existing rating.
5. The system shall allow authenticated users to submit, edit, and delete their own reviews.
6. The system shall prevent users from editing or deleting another user's rating or review.
7. The system shall expose a movie catalogue via REST API supporting pagination and search.
8. The system shall compute or retrieve each movie's average rating from stored rating data.
9. The system shall generate personalized recommendations using content-based filtering, TF-IDF + cosine similarity, and item-based collaborative filtering combined through a hybrid scoring model.
10. The system shall provide a fallback (cold-start) recommendation strategy for users with no rating history.
11. The system shall expose Cine Digest content via a REST API independent of the recommendation module.
12. The system shall reference trailers via an external video identifier (e.g., YouTube key) rather than storing video files.
13. The system shall reference poster images via a URL/path rather than storing binary image data in MySQL.

11. Non-Functional Requirements

Category	Requirement
Security	Passwords hashed (never plain text); authorization enforced per-user for ratings/reviews; secrets never exposed in frontend code
Performance	Movie listing must support pagination/lazy loading rather than returning the entire catalogue at once
Scalability	Recommendation computations (TF-IDF, similarity matrices) should be precomputable/cacheable rather than recalculated per request where feasible
Maintainability	Layered Spring Boot architecture (controller/service/repository/entity/dto) with thin controllers and business logic isolated in services
Portability	Frontend remains framework-free (HTML/CSS/vanilla JS) so it can run without a build pipeline
Data Integrity	Foreign keys and unique constraints enforced at the database level, not only in application code
Availability	Cine Digest must degrade gracefully (cached/fallback content) if the external data provider is unavailable
Usability	Figma design is the visual source of truth; UI structure should not be reinvented ad hoc

12. Technology Stack

Layer	Technology	Notes
Frontend	HTML5, CSS3, Vanilla JavaScript	No frameworks (no React) unless explicitly approved
Frontend Design Source	Figma	Visual source of truth
Frontend Tooling (AI-assisted)	Antigravity, AI coding agents	Used to implement Figma design faithfully
Backend Language	Java	
Backend Framework	Spring Boot	Spring Web, Spring Data JPA

ORM	Hibernate (via Spring Data JPA)	
Build Tool	Maven	
Database	MySQL	
DB Tooling	MySQL Workbench	
Security	Spring Security	For authentication/authorization
Auth Token Strategy	JWT (stateless)	See Decision Pending for specifics
Backend IDE	IntelliJ IDEA	Primary backend development environment
External Movie Data	TMDb (metadata), MovieLens (historical ratings)	See Section 22

Explicitly prohibited without approval: React, Node.js/Express.js as the backend, MongoDB, Firebase, Python as the main backend, deep learning frameworks (TensorFlow, PyTorch), and non-approved ML algorithms (Random Forest, XGBoost, CNN, LSTM) as substitutes for the finalized recommendation algorithms.

13. Frontend Architecture

Responsibilities of the frontend:

- Render all pages (HTML/CSS structure following Figma)
- Call REST APIs and handle responses (success and error states)
- Dynamically render movie cards from JSON movie arrays
- Handle UI interactions: star rating selection, search input, navigation, modals (trailer)
- Display reviews and personalized recommendations returned by the backend
- Render Cine Digest content
- Manage authentication-related frontend flows (storing/attaching auth token, redirecting unauthenticated users away from gated actions)

Explicit non-responsibilities:

- The frontend does not compute recommendations, similarity scores, or average ratings

— these are backend/database concerns.

- The frontend never contains large numbers of manually authored movie cards; it contains exactly **one** reusable card template driven by API data.

Recommended frontend module breakdown (JavaScript):

Module	Responsibility
`api.js`	Centralized `fetch` wrapper for all REST calls, base URL config, auth header injection
`auth.js`	Signup/login/logout, token storage, auth-state helpers
`movies.js`	Fetching movie lists/details, rendering movie cards, search
`ratings.js`	Star rating widget logic, submit/update rating calls
`reviews.js`	Submitting reviews, rendering review lists
`recommendations.js`	Fetching and rendering personalized recommendation cards
`cineDigest.js`	Fetching and rendering Cine Digest content
`ui.js` / `dom.js`	Shared DOM helpers (element creation, templating helpers)

14. Page-Level Requirements

For each page: purpose, main UI components, data required, backend/API dependency, user interactions, authentication requirement, and expected behavior. Exact visuals follow Figma; this section documents functional intent only.

14.1 Login Page

- **Purpose:** Authenticate an existing user.
- **Main components:** Email/username field, password field, submit button, link to Signup.
- **Data required:** Credentials entered by user.
- **API dependency:** `POST /api/auth/login`
- **Interactions:** Submit form, inline validation, error messaging on failed login.
- **Auth requirement:** None (this page is for unauthenticated users).

- **Expected behavior:** On success, store auth token/session and redirect to Home; on failure, show a clear error without revealing whether the email or password was incorrect.

14.2 Signup Page

- **Purpose:** Create a new account.
- **Main components:** Username, email, password (and confirmation) fields, submit button, link to Login.
- **API dependency:** `POST /api/auth/signup`
- **Interactions:** Client-side format validation before submission; server-side validation is authoritative.
- **Auth requirement:** None.
- **Expected behavior:** On success, either auto-login or redirect to Login with a success message (Decision Pending — see below).

> **Decision Pending:** Should signup auto-log the user in (returning a token immediately) or require a separate login step?
>
> - **Recommended:** Auto-login immediately after signup (return a token from the signup endpoint) — reduces friction.
> - **Alternative:** Redirect to Login page requiring explicit credential entry.
> - **Why recommended:** Standard modern UX; avoids an unnecessary redundant step for a brand-new account whose credentials are already known to be correct.

14.3 Home Page

- **Purpose:** Entry point showcasing catalogue highlights and navigation to major sections.
- **Main components:** Navigation bar (Home, Browse, Cine Digest, Recommendations, Profile/Login), featured/movie card grid, search bar.
- **Data required:** A movie list (e.g., popular or a default catalogue slice) from the backend.
- **API dependency:** `GET /api/movies` (paginated)
- **Interactions:** Navigate to other pages, click a movie card to open details, use search.
- **Auth requirement:** None to view; some nav items (Recommendations, Profile) require login to access their content.

14.4 Movie Browsing / Movie Listing Area

- **Purpose:** Let users explore the full catalogue.
- **Main components:** Movie card grid, pagination or lazy-load control, optional genre/filter controls if defined by Figma.
- **API dependency:** `GET /api/movies` with pagination parameters.
- **Interactions:** Scroll/paginate, click a card to view details.
- **Auth requirement:** None.

14.5 Movie Card Component

- See Section 15 for full architectural detail.
- **Purpose:** Reusable UI unit representing one movie in any list context (browsing, search results, recommendations, Cine Digest).
- **Data required per card:** `movieId`, `title`, `posterUrl`, `averageRating` (plus any additional field mandated by the final Figma design).
- **Interactions:** Click → navigate to Movie Details Page.

14.6 Movie Details Page

- **Purpose:** Show full information about a single movie.
- **Main components:** Poster, title, overview, genres, release date, average rating (visual stars), rating input control, trailer button/embed, review list, review submission form.
- **Data required:** Movie object, aggregate rating, reviews list, current user's existing rating (if authenticated).
- **API dependency:** `GET /api/movies/{movieId}`, `GET /api/movies/{movieId}/reviews`, `GET /api/movies/{movieId}/ratings` (aggregate), rating/review submission endpoints.
- **Interactions:** Select a star rating, submit/update rating, submit a review, play trailer.
- **Auth requirement:** Viewing is open to all; rating and reviewing require login.

14.7 Rating Interface

- **Purpose:** Let an authenticated user assign or update a 1.0–5.0 (0.5-step) rating.
- **Main components:** Star selector supporting half-star precision.
- **API dependency:** `POST /api/ratings` (create or update — see Section 27).

- **Interactions:** Click/tap to select a star value; frontend validates increment before sending.
- **Auth requirement:** Required. Unauthenticated users seeing this control should be prompted to log in.

14.8 Review Section

- **Purpose:** Display and collect reviews for a movie.
- **Main components:** Review submission textarea + submit button (authenticated only), list of existing reviews (author, text, date).
- **API dependency:** `POST /api/reviews`, `GET /api/movies/{movieId}/reviews`, `PUT` / `DELETE /api/reviews/{reviewId}` for the review's own author.
- **Auth requirement:** Reading is open; writing/editing/deleting requires login and ownership.

14.9 Personalized Recommendations Section/Page

- **Purpose:** Show movies recommended specifically for the logged-in user.
- **Main components:** Movie card grid populated from the recommendation API; empty/cold-start state messaging when the user has no rating history yet.
- **API dependency:** `GET /api/recommendations/me`
- **Auth requirement:** Required.

14.10 User Profile Area

- **Purpose:** Show the logged-in user's own ratings and reviews; basic account info.
- **Main components:** Account details (username/email), list of the user's ratings, list of the user's reviews.
- **API dependency:** `GET /api/users/{userId}/ratings` (self-scoped), reviews equivalent.
- **Auth requirement:** Required; a user may only view/manage their own profile data.

14.11 Cine Digest Page

- **Purpose:** Present cinema-wide trends and discovery content.
- **Main components:** Sectioned content (Trending, Popular, Upcoming, New Trailers, News/Announcements) rendered using the same movie card template where applicable.

- **API dependency:** `GET /api/cine-digest`
- **Auth requirement:** None.

14.12 Search Results Interface

- **Purpose:** Show movies matching a search query.
- **Main components:** Result grid using the shared movie card template, “no results” state.
- **API dependency:** `GET /api/movies/search?query=...`
- **Auth requirement:** None.

15. Dynamic Movie Card Architecture

This is one of the most important architectural constraints in CineSuggest: **the frontend must never contain thousands of manually authored movie cards.** Instead, there is exactly one reusable card template, and JavaScript populates it dynamically from API data.

Data flow:

```
MySQL Database
   → Spring Boot (Repository → Service)
   → REST API (Controller, JSON response)
   → JavaScript (fetch + JSON parsing)
   → Dynamic movie card creation (loop over array)
   → Rendered movie card grid in the DOM
```

Conceptual movie object contract used by the card renderer:

```
{
  "movieId": 4102,
  "title": "Dune",
  "posterUrl": "https://image.tmdb.org/t/p/w500/xxxxxx.jpg",
  "averageRating": 4.5
}
```

Conceptual rendering logic (illustrative, not final implementation):

```
function renderMovieCards(movies, containerEl) {
  containerEl.innerHTML = "";
  movies.forEach(movie => {
    const card = createMovieCardElement(movie); // one shared template function
    containerEl.appendChild(card);
```

```
  });
}

function createMovieCardElement(movie) {
  const card = document.createElement("div");
  card.className = "movie-card";
  card.dataset.movieId = movie.movieId;
  card.innerHTML = `
    <img src="${movie.posterUrl}" alt="${movie.title} poster" />
    <h3>${movie.title}</h3>
    <div class="rating-stars" data-rating="${movie.averageRating}"></div>
  `;
  card.addEventListener("click", () => navigateToMovieDetails(movie.movieId));
  return card;
}
```

If the API returns 1,000 movies, this same function produces 1,000 cards without any additional hand-authored markup. This is why **pagination or lazy loading is a required production consideration** (see Section 48) — returning the entire catalogue in a single response is inefficient regardless of how efficiently the frontend renders it.

16. Backend Architecture

Backend responsibilities:

- User authentication and account management
- Movie data access and search
- Rating creation/update and average rating computation
- Review creation/read/update/delete
- Recommendation engine execution (content-based, collaborative, hybrid)
- Cine Digest integration with external data
- REST API exposure, input validation, and error handling
- All database communication (the frontend never talks to MySQL directly)

Recommended layered architecture (per package):

Package	Responsibility
`controller`	REST endpoints; parses requests, delegates to services, returns responses. Contains no business logic.

`service`	Core business logic: rating rules, review rules, recommendation orchestration, average calculations.
`repository`	Spring Data JPA interfaces for database access. No business logic.
`model` / `entity`	JPA entities mapped to MySQL tables (`User` , `Movie` , `Rating` , `Review`).
`dto`	Request/response data transfer objects, decoupling API contracts from entity structure.
`config`	Spring configuration: CORS, beans, application-level settings.
`security`	Spring Security configuration, JWT filter/provider, password encoding setup.
`exception`	Custom exceptions and centralized exception handling (`@ControllerAdvice`).
`recommendation`	Recommendation engine: content-based service, TF-IDF service, cosine similarity service, collaborative filtering service, hybrid service.
`integration`	External API clients (TMDb / cinema data provider) feeding Cine Digest and dataset import.

Design principle: keep controllers thin. A controller method should validate the request shape, call exactly one service method, and map the result to a response DTO — no direct repository calls, no recommendation math, no business rules embedded in the controller.

17. Spring Boot Package Structure

```
src/main/java/com/cinesuggest/

├── controller/
│   ├── AuthController.java
│   ├── MovieController.java
│   ├── RatingController.java
│   ├── ReviewController.java
│   ├── RecommendationController.java
│   └── CineDigestController.java
│
├── service/
│   ├── AuthService.java
│   ├── MovieService.java
```

```
│   ├── RatingService.java
│   ├── ReviewService.java
│   ├── RecommendationService.java
│   └── CineDigestService.java
│
├── repository/
│   ├── UserRepository.java
│   ├── MovieRepository.java
│   ├── RatingRepository.java
│   └── ReviewRepository.java
│
├── entity/
│   ├── User.java
│   ├── Movie.java
│   ├── Rating.java
│   └── Review.java
│
├── dto/
│   ├── request/  (e.g., SignupRequest, LoginRequest, RatingRequest, ReviewReques
│   └── response/ (e.g., MovieResponse, RatingResponse, ReviewResponse, Recommend
│
├── security/
│   ├── JwtTokenProvider.java
│   ├── JwtAuthenticationFilter.java
│   └── SecurityConfig.java
│
├── config/
│   └── CorsConfig.java
│
├── exception/
│   ├── GlobalExceptionHandler.java
│   ├── ResourceNotFoundException.java
│   └── InvalidRatingException.java
│
├── recommendation/
│   ├── ContentBasedRecommendationService.java
│   ├── TfIdfService.java
│   ├── CosineSimilarityService.java
│   ├── ItemBasedCollaborativeFilteringService.java
│   └── HybridRecommendationService.java
│
└── integration/
    ├── TmdbClient.java
    └── CineDigestProviderClient.java
```

> These class names are **recommended**, not mandatory. An AI agent may adapt naming to

> match existing conventions in the repository, but should preserve the underlying separation of concerns (thin controllers, business logic in services, data access in repositories, recommendation math isolated in the `recommendation` package).

18. Database Architecture

Technology: MySQL, administered via MySQL Workbench.

Core tables: `users`, `movies`, `ratings`, `reviews`.

Guiding principles:

- Passwords are always hashed, never stored in plain text.
- Poster images and trailer videos are referenced by URL/key — binary media is never stored in MySQL.
- Foreign keys enforce referential integrity between ratings/reviews and their parent user/movie rows.
- A unique constraint on `(user_id, movie_id)` in `ratings` guarantees one active rating per user per movie.

19. Detailed Table Schemas

19.1 `users`

Column	Type	Constraints
user_id	BIGINT	PRIMARY KEY, AUTO_INCREMENT
username	VARCHAR(50)	NOT NULL, UNIQUE
email	VARCHAR(255)	NOT NULL, UNIQUE
password_hash	VARCHAR(255)	NOT NULL
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP
updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP

```
CREATE TABLE users (
    user_id       BIGINT AUTO_INCREMENT PRIMARY KEY,
```

```
    username      VARCHAR(50)  NOT NULL UNIQUE,
    email         VARCHAR(255) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);
```

19.2 `movies`

Column	Type	Constraints / Notes
`movie_id`	`BIGINT`	`PRIMARY KEY` , `AUTO_INCREMENT` — internal CineSuggest ID
`tmdb_id`	`BIGINT`	`UNIQUE` , nullable — external TMDb identifier
`movielens_id`	`BIGINT`	`UNIQUE` , nullable — external MovieLens identifier
`title`	`VARCHAR(500)`	`NOT NULL`
`overview`	`TEXT`	Nullable — sourced from TMDb
`genres`	`VARCHAR(500)`	Denormalized delimited string (see note below)
`poster_path`	`VARCHAR(500)`	URL or path fragment; never binary image data
`release_date`	`DATE`	Nullable
`release_year`	`INT`	Derived from `release_date` for convenient filtering
`trailer_key`	`VARCHAR(100)`	External video reference (e.g., YouTube key); never a video file
`popularity`	`DECIMAL(10,3)`	Optional, sourced from TMDb, used for Cine Digest/cold start
`created_at` / `updated_at`	`TIMESTAMP`	Standard audit columns

```
CREATE TABLE movies (
    movie_id      BIGINT AUTO_INCREMENT PRIMARY KEY,
```

```
    tmdb_id       BIGINT UNIQUE,
    movielens_id  BIGINT UNIQUE,
    title         VARCHAR(500) NOT NULL,
    overview      TEXT,
    genres        VARCHAR(500),
    poster_path   VARCHAR(500),
    release_date  DATE,
    release_year  INT,
    trailer_key   VARCHAR(100),
    popularity    DECIMAL(10,3),
    created_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
    INDEX idx_movies_title (title),
    INDEX idx_movies_tmdb_id (tmdb_id),
    INDEX idx_movies_movielens_id (movielens_id)
);
```

> **Decision Pending: genre storage model.**
>
> - **Recommended (initial phase):** a single denormalized `genres` column (comma- or pipe-delimited, e.g., `"Sci-Fi|Adventure|Drama"`) — simplest to import from TMDb/MovieLens and sufficient for content-based feature text and basic display.
> - **Alternative:** a normalized `genres` table plus a `movie_genres` join table, enabling genre-based filtering/queries without string parsing.
> - **Why recommended:** the current mandatory scope does not require genre-based filtering UI; a normalized model can be introduced later without breaking the API contract (`genres` can be returned as an array either way).

19.3 `ratings`

Column	Type	Constraints
`rating_id`	`BIGINT`	`PRIMARY KEY , AUTO_INCREMENT`
`user_id`	`BIGINT`	`NOT NULL , FOREIGN KEY → users(user_id)`
`movie_id`	`BIGINT`	`NOT NULL , FOREIGN KEY → movies(movie_id)`
`rating`	`DECIMAL(2,1)`	`NOT NULL , CHECK (rating BETWEEN 1.0 AND 5.0)`

`created_at` / `updated_at`	`TIMESTAMP`	Standard audit columns

```
CREATE TABLE ratings (
    rating_id  BIGINT AUTO_INCREMENT PRIMARY KEY,
    user_id    BIGINT NOT NULL,
    movie_id   BIGINT NOT NULL,
    rating     DECIMAL(2,1) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    CONSTRAINT uq_user_movie_rating UNIQUE (user_id, movie_id),
    CONSTRAINT fk_ratings_user  FOREIGN KEY (user_id)  REFERENCES users(user_id)
    CONSTRAINT fk_ratings_movie FOREIGN KEY (movie_id) REFERENCES movies(movie_id
    CONSTRAINT chk_rating_range CHECK (rating >= 1.0 AND rating <= 5.0),
    INDEX idx_ratings_movie_id (movie_id),
    INDEX idx_ratings_user_id (user_id)
);
```

`DECIMAL(2,1)` natively stores exactly one decimal digit (e.g., `4.5`), matching the required 0.5 increments. Half-step enforcement (rejecting values like `4.3`) must additionally be validated in the backend service layer, since MySQL's `CHECK` constraint here only bounds the range, not the increment. See Section 46.

The `uq_user_movie_rating` unique constraint is what guarantees **one active rating per (user, movie) pair** — a repeat submission must be handled as an `UPDATE`, not a new row. This table is the primary input to item-based collaborative filtering, since the full user-item rating matrix is derived directly from it.

19.4 `reviews`

Column	Type	Constraints
`review_id`	`BIGINT`	`PRIMARY KEY`, `AUTO_INCREMENT`
`user_id`	`BIGINT`	`NOT NULL`, `FOREIGN KEY → users(user_id)`
`movie_id`	`BIGINT`	`NOT NULL`, `FOREIGN KEY → movies(movie_id)`
`review_text`	`VARCHAR(2000)`	`NOT NULL` (max length — see Decision Pending in Section 46)
`created_at` /		

`updated_at`	`TIMESTAMP`	Standard audit columns

```
CREATE TABLE reviews (
    review_id   BIGINT AUTO_INCREMENT PRIMARY KEY,
    user_id     BIGINT NOT NULL,
    movie_id    BIGINT NOT NULL,
    review_text VARCHAR(2000) NOT NULL,
    created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    CONSTRAINT fk_reviews_user  FOREIGN KEY (user_id)  REFERENCES users(user_id)
    CONSTRAINT fk_reviews_movie FOREIGN KEY (movie_id) REFERENCES movies(movie_id
    INDEX idx_reviews_movie_id (movie_id),
    INDEX idx_reviews_user_id (user_id)
);
```

Reviews are **user-generated application content**, distinct from dataset-derived ratings (MovieLens). A user may submit a rating and a review independently — neither requires the other unless a future product decision explicitly couples them.

20. Database Relationships

- **User → Ratings:** one user can create many ratings (`users.user_id` 1—N `ratings.user_id`).
- **Movie → Ratings:** one movie can receive many ratings (`movies.movie_id` 1—N `ratings.movie_id`).
- **User → Reviews:** one user can write many reviews (`users.user_id` 1—N `reviews.user_id`).
- **Movie → Reviews:** one movie can receive many reviews (`movies.movie_id` 1—N `reviews.movie_id`).
- **Referential integrity:** all foreign keys use `ON DELETE CASCADE` so that deleting a user or movie cleanly removes their dependent ratings/reviews rather than leaving orphaned rows (subject to product confirmation — see note below).

> **Decision Pending: cascade vs. soft handling on user/movie deletion.**
>
> - **Recommended:** `ON DELETE CASCADE` as shown above — simplest, keeps the database consistent.
> - **Alternative:** soft-delete users/movies (a `deleted_at` flag) and retain historical

> ratings/reviews for recommendation integrity.
>
> - **Why recommended:** the current mandatory scope has no account-deletion or catalogue-removal feature yet; cascade is the simplest correct default until such a feature is specified.

21. ER Diagram

```
erDiagram
    USERS ||--o{ RATINGS : creates
    USERS ||--o{ REVIEWS : writes
    MOVIES ||--o{ RATINGS : receives
    MOVIES ||--o{ REVIEWS : receives

    USERS {
        bigint user_id PK
        varchar username
        varchar email
        varchar password_hash
        timestamp created_at
        timestamp updated_at
    }
    MOVIES {
        bigint movie_id PK
        bigint tmdb_id
        bigint movielens_id
        varchar title
        text overview
        varchar genres
        varchar poster_path
        date release_date
        int release_year
        varchar trailer_key
        decimal popularity
        timestamp created_at
        timestamp updated_at
    }
    RATINGS {
        bigint rating_id PK
        bigint user_id FK
        bigint movie_id FK
        decimal rating
        timestamp created_at
        timestamp updated_at
    }
```

```
REVIEWS {
    bigint review_id PK
    bigint user_id FK
    bigint movie_id FK
    varchar review_text
    timestamp created_at
    timestamp updated_at
}
```

22. Dataset Architecture

CineSuggest's catalogue is populated from two complementary datasets rather than manual entry:

1. **TMDb** — rich movie metadata (title, overview, genres, poster, release date, trailer reference, popularity) used for display and content-based feature text.
2. **MovieLens** — historical user-movie ratings used to bootstrap and validate item-based collaborative filtering.

These two datasets describe overlapping but not identical sets of movies, and **use different ID systems**, which is why explicit ID mapping (Section 25) is required.

23. TMDb Data Strategy

TMDb-sourced metadata feeds:

- The movie catalogue shown in the frontend (poster, title, overview, genres, release info)
- The text corpus used by content-based filtering / TF-IDF
- Trailer references (video key/URL)
- Popularity signals usable for cold start and Cine Digest

Relevant fields typically obtained from TMDb:

Field	Use
Title	Display, search
Overview	Display, content-based feature text
Genres	Display, content-based feature text
Poster path	Frontend `<img>` source (stored as path/URL, not binary)

Release date	Display, filtering
TMDb ID	External identifier, mapping key
Popularity	Cold start, Cine Digest ranking
Trailer/video key	Trailer playback

Media storage rule: poster images and trailer videos are **never** stored as binary data in MySQL. Posters are stored as a path/URL and rendered directly by the frontend `<img>` tag (optionally via a CDN or TMDb's own image hosting). Trailers are stored as a YouTube video key/URL and rendered via an embedded player or an external link — never as an uploaded video file.

24. MovieLens Data Strategy

MovieLens is used primarily for **historical rating data** to bootstrap and validate item-based collaborative filtering — not for movie metadata display.

Relevant files:

File	Purpose
`movies.csv`	MovieLens's own minimal movie metadata (title, genres) — used mainly for mapping/validation, not primary display data
`ratings.csv`	Historical user-movie ratings — the core input for collaborative filtering
`links.csv`	Maps MovieLens `movieId` to external `tmdbId` and `imdbId` — **the critical bridge** between the two datasets
`tags.csv`	Optional free-text tags per movie — may enrich content-based feature text if used

The role of `links.csv`: without it, there is no reliable way to know that MovieLens's internal `movieId` for "Dune" corresponds to TMDb's internal `id` for the same film. `links.csv` is therefore consulted during import to populate `movies.movielens_id` and `movies.tmdb_id` on the *same* CineSuggest `movies` row.

25. Movie ID Mapping Strategy

CineSuggest maintains its **own** internal `movie_id` as the canonical primary key. External

identifiers are stored as reference columns, never assumed interchangeable:

Column	Source	Purpose
`movie_id`	Generated by CineSuggest (`AUTO_INCREMENT`)	Canonical internal ID used throughout the app and APIs
`tmdb_id`	TMDb	Used to re-fetch/refresh metadata from TMDb; also the join key from `links.csv`
`movielens_id`	MovieLens `movies.csv` / `links.csv`	Used to join historical `ratings.csv` rows to the correct CineSuggest movie

Explicit rule: `tmdb_id` and `movielens_id` are **not** the same numbering system and must never be conflated. All joins between the two datasets happen through `links.csv` , and the result is stored on the unified `movies` row so that downstream code (ratings, reviews, recommendations) only ever needs to reason about the single internal `movie_id` .

26. Data Import Strategy

Conceptual pipeline:

```
Dataset CSV files (TMDb export + MovieLens movies.csv/ratings.csv/links.csv)
   → CSV inspection & data cleaning (encoding, null/duplicate handling)
   → ID mapping (movielens_id ↔ tmdb_id via links.csv)
   → MySQL import (populate movies table; populate ratings table with historical
   → Spring Boot repositories read the imported data
   → REST APIs expose it
   → Frontend renders it
```

Import steps to document/perform:

1. **CSV inspection** — confirm column layouts, encodings, and row counts for `movies.csv` , `ratings.csv` , `links.csv` , and the TMDb export.
2. **Data cleaning** — trim whitespace, normalize genre delimiters, handle missing overviews/posters gracefully (nullable fields).
3. **Duplicate handling** — deduplicate by `tmdb_id` / `movielens_id` before insert; a movie must not be inserted twice under two different `movie_id` s.
4. **Null value handling** — nullable metadata (e.g., missing poster) should not block import; the frontend must handle a missing poster/trailer gracefully (see Error Handling).

5. **ID mapping** — join MovieLens `movieId` to `tmdbId` via `links.csv` before inserting into the unified `movies` table.
6. **Import execution** — batch insert into MySQL (via a one-time import script/job, not manual entry).
7. **Post-import validation** — row counts match expectations, no orphaned foreign keys, spot-check known titles.

Critical distinction — historical vs. live ratings:

	Historical MovieLens Ratings	**New CineSuggest User Ratings**
Origin	Imported dataset, represents past MovieLens users	Real CineSuggest account holders rating through the app
Purpose	Bootstraps item-item similarity for collaborative filtering before CineSuggest has enough of its own rating volume	Drives live personalization and the ongoing rating/average-rating features
Storage	May be imported into the `ratings` table (Decision Pending below) or kept in a separate offline dataset used only to precompute item-item similarity	Always stored in the `ratings` table under the acting user's real `user_id`

> **Decision Pending: should historical MovieLens ratings be imported directly into the live `ratings` table (under synthetic "dataset" user rows), or kept as a separate offline corpus used only to precompute item-item similarity?**
>
> - **Recommended:** keep historical MovieLens ratings in a **separate offline dataset/table** (e.g., `movielens_ratings_raw`) used exclusively to precompute the item-item similarity matrix for collaborative filtering. Do not mix them into the live `ratings` table, which should represent only real CineSuggest account activity (average ratings shown to users should reflect real users, not synthetic dataset rows).
> - **Alternative:** import MovieLens ratings into `ratings` under placeholder "seed" user accounts, simplifying the collaborative filtering query (one table to scan) at the cost of polluting "average rating" displays with historical data not generated by real CineSuggest users.

> - **Why recommended:** keeps "average rating shown to users" honest (real CineSuggest users only) while still letting collaborative filtering benefit from MovieLens's much larger historical volume during cold start.

27. Rating System

Requirements: ratings range from **1.0 to 5.0** in **0.5 increments** (valid values: 1.0, 1.5, 2.0, 2.5, 3.0, 3.5, 4.0, 4.5, 5.0). Stored as `DECIMAL(2,1)`. One active rating per `(user_id, movie_id)` pair — re-rating **updates** the existing row rather than creating a duplicate.

Full lifecycle (example: user rates *Dune* 4.5 stars):

1. User logs in (obtains an auth token).
2. User opens the *Dune* details page.
3. User selects **4.5** stars on the rating widget.
4. Frontend validates the value is within range and on a 0.5 step before sending.
5. Frontend sends the rating via `POST /api/ratings` with the auth token attached.
6. Spring Boot authenticates the request (valid token → known user).
7. Spring Boot (service layer) validates the rating value server-side (authoritative check — never trust the client).
8. Spring Boot checks whether a rating already exists for `(user_id, movie_id)`.
    - If it exists → **update** the existing row's `rating` and `updated_at`.
    - If not → **insert** a new row.
9. MySQL persists the change.
10. The movie's average rating is recalculated or retrieved (see below).
11. This rating becomes an input row for the item-based collaborative filtering user-item matrix going forward.

Average rating calculation: computed as the arithmetic mean of all rows in `ratings` for a given `movie_id`.

```
SELECT movie_id, AVG(rating) AS average_rating, COUNT(*) AS rating_count
FROM ratings
WHERE movie_id = ?
GROUP BY movie_id;
```

> **Decision Pending: compute average rating on-demand vs. maintain a cached/denormalized column.**
>
> - **Recommended (initial phase):** compute on-demand via the `AVG()` query above, backed by the `idx_ratings_movie_id` index — simplest, always consistent, and fast enough at moderate scale.
> - **Alternative:** add a denormalized `average_rating` / `rating_count` pair on the `movies` table, updated whenever a rating is created/updated, to avoid an aggregate query on every movie-details request.
> - **Why recommended:** avoids a second write path (and possible drift) until real usage data shows the on-demand query is a bottleneck; revisit under Performance Considerations.

Why ratings matter beyond display: the `ratings` table is the **primary input to item-based collaborative filtering** — the user-item matrix used to compute item-item similarity is built directly from these rows. Without sufficient rating volume, collaborative filtering degrades toward the cold-start strategy.

28. Review System

Full lifecycle:

1. User logs in.
2. User opens a movie's details page.
3. User enters review text into the review form.
4. Frontend submits the review via `POST /api/reviews` with the auth token attached.
5. Backend validates the request (non-empty text, within max length, valid `movie_id`, authenticated user).
6. Review is persisted in `reviews`.
7. The movie details page retrieves the review list via `GET /api/movies/{movieId}/reviews`, typically **sorted by most recent first** (default — see Decision Pending below).

> **Decision Pending: default review sort order.**
>
> - **Recommended:** most recent first (`ORDER BY created_at DESC`).
> - **Alternative:** highest-rated-reviewer-first, or a helpfulness/likes-based sort (requires

> the “review likes” future enhancement, not in current scope).
>
> - **Why recommended:** simplest, requires no additional feature, and matches common review-list UX conventions.

Ratings and reviews are independent: a user may submit either, both, or neither for a given movie unless a future product decision explicitly requires pairing them. A review does not imply or require a numeric rating, and vice versa.

Ownership enforcement: `PUT` / `DELETE /api/reviews/{reviewId}` must verify that the authenticated user’s `user_id` matches the review’s `user_id` before allowing modification — enforced in the service layer, not just hidden in the frontend.

29. Trailer System

CineSuggest supports official movie trailers via an **external video reference** — a YouTube video key/URL (or equivalent), stored in `movies.trailer_key`. Full video files are never stored in MySQL.

Frontend behavior options:

- A “Watch Trailer” button that opens a modal with an embedded YouTube player, or
- A button that opens the official trailer in a new tab if embedding is unavailable/blocked.

> **Decision Pending: embed vs. external-link trailer playback.**
>
> - **Recommended:** embed via the YouTube IFrame Player using the stored `trailer_key`, inside an in-page modal — keeps the user inside CineSuggest.
> - **Alternative:** open the trailer in a new browser tab/window pointing directly at YouTube.
> - **Why recommended:** better perceived UX and consistent with modern movie-app conventions; falls back to an external link automatically if no `trailer_key` is present (missing-trailer state — see Error Handling).

Data flow: `movies.trailer_key` (populated during TMDb import) → returned as part of the movie details API response → frontend renders the embed or link using that key. No separate trailer endpoint is required unless a future enhancement demands multiple trailers per movie.

30. Authentication and Security

Core rules:

- Passwords are always hashed before storage (`users.password_hash`) — plain text is never persisted, logged, or returned by any API.
- Spring Security governs authentication/authorization.
- JWT provides stateless authentication for the REST API.

Conceptual flow:

```
Login credentials (email/username + password)
   → Backend validation (Spring Security AuthenticationManager)
   → Password verified against stored hash
   → JWT issued on success
   → Frontend stores the token (see Decision Pending) and attaches it to subseque
   → Backend verifies the token on each protected request (JwtAuthenticationFilte
```

Authorization rules:

- A user may only update or delete **their own** reviews.
- A user may only create/update **their own** rating (the `user_id` is derived from the authenticated principal, never taken from client-supplied input).
- Personalized recommendation endpoints should be scoped to "the current authenticated user" (e.g., `/api/recommendations/me`) rather than accepting an arbitrary `userId` path parameter, to avoid leaking one user's personalization to another.

> **Decision Pending: password hashing algorithm/parameters.**
>
> - **Recommended:** BCrypt (Spring Security's `BCryptPasswordEncoder`) with a cost factor of 10–12.
> - **Alternative:** Argon2 (`Argon2PasswordEncoder`), which is more modern but heavier and less universally defaulted-to in Spring Security tutorials/tooling.
> - **Why recommended:** BCrypt is the Spring Security default, well-understood, and sufficient for this project's threat model.

> **Decision Pending: where the frontend stores the JWT.**
>
> - **Recommended:** an in-memory JS variable plus a short-lived, `httpOnly` cookie issued by the backend for session persistence across reloads, avoiding `localStorage` exposure to XSS.
> - **Alternative:** `localStorage` — simpler to implement in a framework-free frontend, but more exposed to XSS token theft.

> - **Why recommended:** stronger security posture; the alternative remains acceptable for an early/internal build if the team prioritizes implementation speed over hardening at this stage.

Input validation: every write endpoint (signup, login, ratings, reviews) performs server-side validation regardless of any client-side checks already performed (see Section 46).

CORS: because the vanilla JS frontend and the Spring Boot backend will very likely run on different ports during development (e.g., a static file server on one port, Spring Boot on `8080`), CORS must be explicitly configured (allowed origins, methods, headers) in the `config` / `security` layer — this is not optional even for local development.

31. REST API Architecture

All communication between frontend and backend happens over REST endpoints exchanging JSON. The frontend never queries MySQL directly.

Endpoint groups:

Group	Base Path
Authentication	`/api/auth`
Movies	`/api/movies`
Ratings	`/api/ratings`, nested under `/api/movies/{movieId}/ratings` and `/api/users/{userId}/ratings` for reads
Reviews	`/api/reviews`, nested under `/api/movies/{movieId}/reviews` for reads
Recommendations	`/api/recommendations`
Cine Digest	`/api/cine-digest`

Security note on user-scoped endpoints: exposing arbitrary `userId` values in personalized endpoints (e.g., `GET /api/users/{userId}/ratings`) should be avoided or carefully authorized — prefer deriving the acting user from the authenticated token (`/api/users/me/ratings` or `/api/recommendations/me`) rather than trusting a client-supplied ID.

32. Detailed API Specification

For each endpoint: purpose, method, auth requirement, request, response, and the primary backend service involved.

32.1 `POST /api/auth/signup`

- **Purpose:** Create a new user account.
- **Auth:** None required.
- **Service:** `AuthService`

Request:

```
{
  "username": "moviefan92",
  "email": "moviefan92@example.com",
  "password": "SecurePass123!"
}
```

Response (`201 Created`):

```
{
  "userId": 501,
  "username": "moviefan92",
  "email": "moviefan92@example.com",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

32.2 `POST /api/auth/login`

- **Purpose:** Authenticate an existing user and issue a token.
- **Auth:** None required.
- **Service:** `AuthService`

Request:

```
{ "email": "moviefan92@example.com", "password": "SecurePass123!" }
```

Response (`200 OK`):

```
{ "userId": 501, "username": "moviefan92", "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6
```

32.3 `GET /api/movies`

- **Purpose:** Paginated movie catalogue listing.
- **Auth:** None required.
- **Service:** `MovieService`

Request (query params): `?page=0&size=24`

Response (`200 OK`):


```
{
  "page": 0,
  "size": 24,
  "totalElements": 4213,
  "totalPages": 176,
  "movies": [
    { "movieId": 4102, "title": "Dune", "posterUrl": "https://image.tmdb.org/t/p/
  ]
}
```


32.4 `GET /api/movies/{movieId}`

- **Purpose:** Full movie detail.
- **Auth:** None required.
- **Service:** `MovieService`

Response (`200 OK`):


```
{
  "movieId": 4102,
  "title": "Dune",
  "overview": "Paul Atreides, a brilliant and gifted young man...",
  "genres": ["Sci-Fi", "Adventure", "Drama"],
  "posterUrl": "https://image.tmdb.org/t/p/w500/xxxxxx.jpg",
  "releaseDate": "2021-10-22",
  "averageRating": 4.5,
  "ratingCount": 812,
  "trailerKey": "8g18jFHCLXk"
}
```


32.5 `GET /api/movies/search`

- **Purpose:** Search movies by title.

- **Auth:** None required.
- **Service:** `MovieService`

Request: `?query=dune&page=0&size=24` Response: same shape as `GET /api/movies`.

32.6 `GET /api/movies/trending`

- **Purpose:** Optional convenience endpoint for a trending slice of the catalogue, distinct from full Cine Digest content.
- **Auth:** None required.
- **Service:** `MovieService` (or delegates into `CineDigestService` — Decision Pending on ownership boundary).

> **Decision Pending:** should `/api/movies/trending` exist as a thin catalogue-side convenience, or should all trend-related data flow exclusively through `/api/cine-digest`?
>
> - **Recommended:** omit `/api/movies/trending` from the mandatory build and route all trend content through `/api/cine-digest`, keeping exactly one place responsible for "what's trending."
> - **Alternative:** keep a lightweight `/api/movies/trending` backed by internal popularity data (not the external Cine Digest provider) for fast catalogue-only trending without an external API dependency.

32.7 `POST /api/ratings`

- **Purpose:** Create or update the authenticated user's rating for a movie.
- **Auth:** Required.
- **Service:** `RatingService`

Request:

```
{ "movieId": 4102, "rating": 4.5 }
```

Response (`200 OK`):

```
{ "ratingId": 9981, "movieId": 4102, "rating": 4.5, "updated": true }
```

32.8 `PUT /api/ratings/{ratingId}`

- **Purpose:** Explicitly update a specific rating by ID (alternative to the upsert behavior of `POST /api/ratings`).
- **Auth:** Required; caller must own the rating.
- **Service:** `RatingService`

32.9 `GET /api/movies/{movieId}/ratings`

- **Purpose:** Aggregate rating info for a movie (average + count); not a per-user listing.
- **Auth:** None required.
- **Service:** `RatingService`

Response:

```
{ "movieId": 4102, "averageRating": 4.5, "ratingCount": 812 }
```

32.10 `GET /api/users/me/ratings`

- **Purpose:** The authenticated user's own rating history (used by Profile and as recommendation input visibility).
- **Auth:** Required; self-scoped only.
- **Service:** `RatingService`

32.11 `POST /api/reviews`

- **Purpose:** Submit a new review.
- **Auth:** Required.
- **Service:** `ReviewService`

Request:

```
{ "movieId": 4102, "reviewText": "A visually stunning adaptation that respects th
```

Response (`201 Created`):

```
{ "reviewId": 3321, "movieId": 4102, "userId": 501, "reviewText": "A visually stu
```

32.12 `GET /api/movies/{movieId}/reviews`

- **Purpose:** List reviews for a movie (default sort: most recent first).
- **Auth:** None required.
- **Service:** `ReviewService`

32.13 `PUT /api/reviews/{reviewId}`

- **Purpose:** Edit an existing review.
- **Auth:** Required; caller must own the review.
- **Service:** `ReviewService`

32.14 `DELETE /api/reviews/{reviewId}`

- **Purpose:** Delete an existing review.
- **Auth:** Required; caller must own the review.
- **Service:** `ReviewService`

32.15 `GET /api/recommendations/me`

- **Purpose:** Personalized recommendations for the authenticated user.
- **Auth:** Required.
- **Service:** `RecommendationService` (delegates to `HybridRecommendationService`)

Response:

```
{
  "userId": 501,
  "strategy": "hybrid",
  "recommendations": [
    { "movieId": 5310, "title": "Arrival", "posterUrl": "https://image.tmdb.org/t
  ]
}
```

`strategy` reports which path produced the list (`"hybrid"`, or `"cold-start"` for new users — see Section 39), which is useful for both frontend messaging and debugging.

32.16 `GET /api/cine-digest`

- **Purpose:** Retrieve current cinema trend/discovery content.
- **Auth:** None required.

- **Service:** `CineDigestService`

Response:

```
{
  "generatedAt": "2026-07-13T12:00:00Z",
  "trending": [ { "movieId": 4102, "title": "Dune", "posterUrl": "https://image.t
  "upcoming": [ { "movieId": 6120, "title": "Avatar 4", "posterUrl": "https://ima
  "newTrailers": [ { "movieId": 6120, "trailerKey": "abc123XYZ" } ]
}
```

33. Recommendation Engine

CineSuggest's recommendation engine is a **finalized, fixed set of algorithms** that must not be silently replaced with unrelated ML approaches:

1. Content-Based Filtering
2. TF-IDF Vectorization
3. Cosine Similarity
4. Item-Based Collaborative Filtering
5. Hybrid Recommendation System (combining 1–3 with 4)

These execute entirely as **backend application code** — services/classes within the `recommendation` package — never as frontend logic and never manually calculated by a human.

34. Content-Based Filtering

Purpose: recommend movies similar to movies a given user already likes, based on the movies' own textual/categorical features (not other users' behavior).

Feature sources per movie:

- Overview (TMDb)
- Genres
- Keywords/tags (if available, e.g., from MovieLens `tags.csv`)

Combined feature text: for each movie, concatenate the relevant fields into a single text blob (e.g., overview + genres + tags), which becomes the unit that TF-IDF vectorizes. This "combined representation" lets a single vector capture a movie's overall thematic profile

rather than treating each field independently.

How it reflects user preference: a user's liked movies (e.g., rated 4.0+) are used as reference points; CineSuggest finds other movies whose combined feature vectors are most similar (via cosine similarity) to those reference movies, then aggregates/ranks the results.

35. TF-IDF Vectorization

Purpose: convert each movie's combined feature text into a numerical vector so that similarity can be computed mathematically.

```
Movie metadata text  →  TF-IDF  →  Numerical feature vector
```

- **Vocabulary:** the set of all distinct terms appearing across every movie's combined feature text in the catalogue.
- **Term Frequency (TF):** how often a term appears within one movie's own feature text, relative to that document's length.
- **Inverse Document Frequency (IDF):** a weighting that reduces the importance of terms that appear in *many* movies (e.g., generic words like "story," "life," "world") and increases the importance of terms that are distinctive to fewer movies (e.g., "telekinetic," "heist," "Arrakis").
- **Why TF-IDF suits CineSuggest:** it naturally downweights common, low-signal words in overviews while emphasizing genre- and plot-distinctive terms — exactly the kind of signal needed to tell two similarly-themed but distinct movies apart.
- **Output usage:** the resulting per-movie vectors are the direct input to cosine similarity (Section 36); the entire catalogue's vectors can be precomputed once and reused across many recommendation requests (see Performance Considerations).

36. Cosine Similarity

Purpose: quantify how similar two movies are, based on their TF-IDF vectors.

- Compares **Movie A's vector** against **Movie B's vector** by measuring the cosine of the angle between them in feature space.
- A cosine similarity closer to **1** means the two movies are more similar in feature space; closer to **0** means less similar.
- A **similarity matrix** can be built by computing pairwise cosine similarity across all movies (or a relevant subset), so that "most similar movies to X" becomes a fast lookup

rather than a recomputation per request.

- **Selecting top similar movies:** for a given seed movie (or set of a user's liked movies), sort candidate movies by similarity score descending and take the top *N* as content-based candidates.

37. Item-Based Collaborative Filtering

Purpose: recommend movies based on **rating pattern similarity** between movies, independent of textual content — i.e., "movies that tend to be rated similarly by the same users."

Conceptual example: users who rated *Dune* highly also frequently rated *Interstellar* and *Arrival* highly — so *Dune* is considered **behaviorally similar** to those movies, even though the underlying similarity computation never inspects their plots or genres.

User-item rating matrix:

	Movie 1	Movie 2	Movie 3	...
User A	4.5	—	3.0	...
User B	5.0	4.0	—	...
User C	—	4.5	2.5	...

- **Rows:** users. **Columns:** movies. **Values:** ratings (blank/ – = not rated).
- **Item-item similarity:** computed from the columns of this matrix — two movies are similar if they tend to receive similar ratings from the same users.
- **Sparse data:** most user-movie pairs are unrated (a real user rates a tiny fraction of the catalogue), so the matrix is overwhelmingly sparse; similarity computations must tolerate and expect this rather than assuming dense data.
- **Rating patterns / neighborhood selection:** for a target movie, its "neighborhood" is the set of other movies with the highest item-item similarity scores, computed from overlapping rater behavior.
- **Weighted recommendation scoring (conceptual):** a candidate movie's collaborative score for a user can be estimated as a similarity-weighted combination of the user's own ratings on movies in that candidate's neighborhood — movies more similar to things the user already rated highly score higher.
- **Why user ratings are essential:** without real rating volume (from live CineSuggest

users and/or historical MovieLens data), this entire component has no signal to work from — it is a direct consequence of the `ratings` table's completeness.

38. Hybrid Recommendation System

CineSuggest combines the **Content-Based Score** and the **Collaborative Filtering Score** into one final ranking:

```
Final Recommendation Score =
    (Content Weight × Content-Based Score)
  + (Collaborative Weight × Collaborative Filtering Score)
```

> **Decision Pending: final hybrid weights.**
>
> - **Recommended (initial testing default):** 70% content-based / 30% collaborative (`0.7` / `0.3`), because early on the collaborative signal will be sparse (few real CineSuggest ratings) while content-based signal is available immediately from TMDb metadata.
> - **Alternative:** a 50/50 or 40/60 split favoring collaborative filtering once sufficient rating volume accumulates, or a dynamically adjusted weight based on how many ratings the user/catalogue currently has.
> - **Why recommended, and why not final:** weights must be configurable (e.g., an application property, not a hardcoded literal) and revisited empirically as real usage data accumulates — the master document intentionally avoids fixing this permanently per the project's own instructions.

Ranking and filtering:

- Movies the user has **already rated** are typically excluded from their own recommendation list (they've already expressed an opinion on it).
- Candidates are ranked by final hybrid score, descending.
- The **Top N** movies (N configurable, e.g., 20) are returned by the recommendation API.

39. Cold Start Strategy

Problem: a newly registered user has no rating history, so collaborative filtering has nothing to work from, and even content-based filtering has no "liked movies" to anchor against.

Recommended cold-start strategy:

1. On signup (or first visit to Recommendations), optionally prompt the user to rate a small number of movies or select a few genre preferences.
2. Until sufficient ratings exist, serve **popular/trending movies** (using TMDb `popularity` or Cine Digest data) as a reasonable default recommendation set.
3. As soon as the user has rated even a handful of movies, shift toward **content-based** recommendations first (content-based needs only a few reference movies to function), phasing in collaborative filtering as rating volume grows.
4. The recommendation API response's `strategy` field (see 32.15) should report `"cold-start"` so the frontend can, if desired, show a "Rate a few movies to improve your recommendations" prompt.

Cold-start recommendations are explicitly distinguished from personalized hybrid recommendations both in this strategy and in the API response's `strategy` field — they are a fallback, not a silent substitute.

40. Recommendation Execution Flow

```
User logs in
   → User's rating history is retrieved
   → User preference signals are identified (liked movies, genre patterns)
   → Content-based scores are generated (TF-IDF + cosine similarity against liked
   → Collaborative scores are generated (item-based similarity against the user-i
   → Scores are normalized (if on different scales)
   → Hybrid scoring combines the two signals using the configured weights
   → Movies already rated by the user are filtered out
   → Remaining candidates are ranked by final score
   → Top N movies are selected
   → Recommendation API returns the ranked movie list
   → Frontend dynamically renders recommendation cards using the shared movie car
```

This entire flow executes as backend application code (services within the `recommendation` package); it is never a frontend computation and never manually calculated.

41. Cine Digest Architecture

Conceptual data flow:

```
External cinema/movie data source (e.g., TMDb trending/now-playing endpoints)
   → Spring Boot integration service (integration/CineDigestProviderClient)
   → Data transformation (map external schema → CineSuggest's Cine Digest respons
```

→ Cine Digest REST API (GET /api/cine-digest)
→ Frontend Cine Digest page (rendered via the shared movie card template where

Why the backend calls the external API (not the frontend):

- API keys/secrets for the external provider must never be exposed in frontend JavaScript.
- The backend can apply caching, rate-limit handling, and fallback logic in one place rather than duplicating that logic (and the secret) across every client.

Caching: Cine Digest content changes on the order of hours, not seconds — the backend should cache the transformed response for a configurable interval (e.g., 30–60 minutes) rather than calling the external provider on every page load.

Rate limits: external providers typically cap request volume; caching directly mitigates this, and the integration client should handle provider errors (e.g., HTTP 429) without propagating a raw failure to the frontend.

Fallback behavior: if the external provider is unavailable, Cine Digest should serve the **last successfully cached response** (with a note that it may be stale) rather than showing a broken page; only if no cached data exists at all should the frontend show an explicit "Cine Digest is temporarily unavailable" state.

Conceptual separation restated: Cine Digest never reads the `ratings` table for a specific user and never runs the recommendation engine — it is sourced from an external cinema data feed and reflects the wider cinema landscape, not any individual's taste profile.

> **Decision Pending: which external provider powers Cine Digest.**
>
> - **Recommended:** reuse TMDb's own trending/popular/upcoming/now-playing endpoints, since TMDb is already the metadata source of truth for the catalogue — this avoids introducing a second external vendor and a second ID-mapping problem.
> - **Alternative:** a dedicated cinema news API for awards/announcements content that TMDb doesn't cover, layered on top of TMDb for the movie-list portions.
> - **Why recommended:** minimizes new integration surface area for the initial build; the news/announcements category can be added later as an additional, optional integration.

42. External API Integration

Integration	Used By	Notes
	Catalogue import (Section 23),	API key held server-side only; never

TMDb	Cine Digest (Section 41)	shipped to the frontend
MovieLens dataset	One-time/periodic offline import (Section 24, 26)	Not a live API — a downloaded dataset processed by an import routine

All external HTTP calls happen from the `integration` package on the backend. The frontend never calls TMDb (or any external provider) directly.

43. Complete System Architecture

Text diagram:

```
User
  ↓
Frontend (HTML + CSS + Vanilla JavaScript)
  ↓  REST calls (JSON over HTTP)
Spring Boot Controllers
  ↓
Service Layer  ──────────────┐
  ↓                          ↓
Repository Layer      Recommendation Engine (recommendation package)
  ↓                          ↓
MySQL  ←──────────────────────
  ↑
Integration Layer (integration package)
  ↓
External Movie/Cinema APIs (TMDb, etc.)
```

Mermaid diagram:

```
flowchart TD
    U[User Browser] --> FE["Frontend: HTML / CSS / Vanilla JS"]
    FE -->|REST + JSON| CTRL[Spring Boot Controllers]
    CTRL --> SVC[Service Layer]
    SVC --> REPO["Repository Layer (Spring Data JPA)"]
    SVC --> REC["Recommendation Engine (Content-Based, TF-IDF, Cosine Similarity,
    SVC --> INTG[Integration Layer]
    REPO --> DB[(MySQL)]
    REC --> DB
    INTG --> EXT[("External Movie / Cinema API - e.g. TMDb")]
```

44. Data Flow Diagrams

Two representative flows are shown as sequence diagrams; the remaining flows are documented as numbered steps in Section 45 to avoid repetitive diagramming.

Rating submission:

```
sequenceDiagram
    participant F as Frontend
    participant C as RatingController
    participant S as RatingService
    participant R as RatingRepository
    participant DB as MySQL

    F->>C: POST /api/ratings {movieId, rating} + JWT
    C->>S: submitRating(userId, movieId, rating)
    S->>S: validate range + 0.5 increment
    S->>R: findByUserIdAndMovieId(userId, movieId)
    R->>DB: SELECT ...
    DB-->>R: existing row or none
    alt rating exists
        S->>R: update existing rating
    else no existing rating
        S->>R: insert new rating
    end
    R->>DB: INSERT/UPDATE
    DB-->>R: OK
    R-->>S: saved Rating
    S-->>C: RatingResponse
    C-->>F: 200 OK { ratingId, rating, updated }
```

Personalized recommendation generation:

```
sequenceDiagram
    participant F as Frontend
    participant C as RecommendationController
    participant H as HybridRecommendationService
    participant CB as ContentBasedRecommendationService
    participant CF as ItemBasedCollaborativeFilteringService
    participant DB as MySQL

    F->>C: GET /api/recommendations/me + JWT
    C->>H: getRecommendations(userId)
    H->>DB: fetch user's rating history
    H->>CB: getContentScores(userId, likedMovies)
    H->>CF: getCollaborativeScores(userId)
    CB-->>H: content scores
    CF-->>H: collaborative scores
```

```
H->>H: combine scores (weighted), filter already-rated, rank, take top N
H-->>C: ranked recommendation list
C-->>F: 200 OK { recommendations }
```

45. Detailed Feature Data Flows

1. **User signup:** frontend form → `POST /api/auth/signup` → `AuthService` validates uniqueness of email/username → password hashed → `User` row inserted → JWT issued → returned to frontend → frontend stores token.
2. **User login:** frontend form → `POST /api/auth/login` → `AuthService` looks up user by email → verifies password hash → JWT issued → returned to frontend.
3. **Movie listing:** frontend requests a page → `GET /api/movies?page=&size=` → `MovieService` → `MovieRepository` paginated query → JSON array of movie summaries → frontend renders cards.
4. **Dynamic movie card generation:** movie array from any listing/search/recommendation/Cine Digest endpoint → JavaScript loop → one shared `createMovieCardElement()` call per movie → cards appended to the DOM (see Section 15).
5. **Movie details retrieval:** user clicks a card → frontend navigates with `movieId` → `GET /api/movies/{movieId}` (+ ratings aggregate + reviews) → rendered details page.
6. **User rating submission:** see the sequence diagram in Section 44.
7. **User rating update:** identical flow to submission — the service layer's existing-row check (Step 8 in Section 27) transparently converts a repeat submission into an update.
8. **Review submission:** frontend form → `POST /api/reviews` → `ReviewService` validates → `Review` row inserted → returned to frontend → review list re-fetched or optimistically updated.
9. **Review retrieval:** movie details page loads → `GET /api/movies/{movieId}/reviews` → `ReviewService` → `ReviewRepository` (sorted by recency) → rendered list.
10. **Personalized recommendation generation:** see the sequence diagram in Section 44.
11. **Cine Digest retrieval:** frontend loads Cine Digest page → `GET /api/cine-digest` → `CineDigestService` returns cached transformed data (refreshing from the external provider if the cache is stale) → frontend renders sectioned content using the shared card template.
12. **Trailer playback:** movie details page already has `trailerKey` from the details response

→ user clicks “Watch Trailer” → frontend opens embed/modal using that key — no additional API call required.

46. Validation Rules

Field	Rule
Rating	`1.0 ≤ rating ≤ 5.0`, must be a multiple of `0.5` — enforced in both the database (`CHECK` on range) and the backend service (range **and** increment)
Review text	Must not be empty/whitespace-only; maximum length enforced (see Decision Pending below)
Email	Must be valid email format; must be unique
Password	Minimum length and complexity (see Decision Pending below)
Movie ID	Must reference an existing `movies` row for any rating/review/detail request
Authentication	Required for: submitting/updating a rating, submitting/editing/deleting a review, viewing personalized recommendations

> **Decision Pending: maximum review length.**
>
> - **Recommended:** 2,000 characters (matches the `VARCHAR(2000)` schema in Section 19.4) — long enough for a substantive review, short enough to keep pages readable and storage predictable.
> - **Alternative:** a shorter limit (e.g., 500 characters) to encourage concise reviews, or a much larger `TEXT` column with no hard cap.
> - **Why recommended:** balances expressiveness with UI/storage predictability; can be revisited without a breaking schema change if switched to `TEXT`.

> **Decision Pending: minimum password requirements.**
>
> - **Recommended:** minimum 8 characters, at least one letter and one number.
> - **Alternative:** stricter NIST-aligned guidance (length-focused, no forced complexity rules, but breach-list checking) — more secure but heavier to implement for an initial build.
> - **Why recommended:** reasonable baseline security without excessive early implementation overhead.

47. Error Handling

Scenario	Frontend Responsibility	Backend Responsibility
Invalid login	Show generic “invalid email or password” message	Return `401 Unauthorized`; never reveal which field was wrong
Duplicate email on signup	Show inline “email already in use” message	Return `409 Conflict` with a clear error code/message
Movie not found	Show a “movie not found” page/state	Return `404 Not Found`
Invalid rating (out of range / bad increment)	Prevent submission client-side where possible	Return `400 Bad Request` regardless of client-side checks (authoritative validation)
Empty review	Disable submit / show inline message	Return `400 Bad Request`
Unauthorized rating/review modification	N/A (should be prevented by only showing edit controls for own content)	Return `403 Forbidden`
External API (TMDb/Cine Digest provider) failure	Show cached/fallback Cine Digest content or a clear “temporarily unavailable” state	Serve last cached response; log the failure; do not crash the endpoint
Database failure	Show a generic “something went wrong” message	Return `500 Internal Server Error`; log details server-side only
No recommendation history (cold start)	Show a friendly “rate a few movies to get recommendations” prompt	Return the cold-start recommendation set with `strategy: "cold-start"` rather than an empty/error response
Missing poster	Render a placeholder poster image	Return `posterUrl: null` rather than a broken/fabricated URL
Missing trailer	Hide/disable the “Watch Trailer” button	Return `trailerKey: null`

48. Performance Considerations

- **Pagination / lazy loading:** `GET /api/movies` and search must always be paginated; returning the entire catalogue in one response is explicitly disallowed at scale.
- **Database indexes:** `movies.title` (search), `ratings.movie_id` / `ratings.user_id`, `reviews.movie_id` / `reviews.user_id` (all foreign keys indexed for join/filter performance) — already reflected in the schemas in Section 19.
- **Rating query optimization:** the average-rating aggregate query relies on the `idx_ratings_movie_id` index; revisit denormalized caching (see Decision Pending in Section 27) if this becomes a hotspot.
- **Recommendation calculation cost:** TF-IDF vectors and the cosine similarity matrix are computed over the *whole catalogue*, which is expensive to redo per request — **precompute and cache** these rather than recalculating on every recommendation call.
- **Caching:** Cine Digest responses should be cached for a configurable interval (see Section 41); recommendation similarity data should be recomputed on a scheduled interval rather than on-demand.
- **Poster delivery:** posters are served via external URL (TMDb's own image hosting/CDN, or a CDN CineSuggest fronts), never re-hosted as binary blobs from MySQL.
- **Recommendation refresh strategy:** full similarity-matrix recomputation is a batch/background job (e.g., nightly or on a schedule), not something triggered synchronously inside a user's request.

49. Testing Strategy

Layer	Approach	Tooling
Backend unit tests	Service-layer logic (rating validation, review validation, recommendation scoring) tested in isolation	JUnit, Spring Boot Test
API tests	Endpoint-level request/response verification, auth enforcement	Spring Boot Test (`MockMvc`), Postman for manual/exploratory testing
Database tests	Constraint verification (unique `user_id` + `movie_id` , foreign key cascade behavior)	Spring Boot Test with a test database/profile
Recommendation	TF-IDF vector correctness on known	

testing	inputs, cosine similarity sanity checks, cold-start fallback triggering correctly	JUnit
Frontend testing	Manual/exploratory testing of card rendering, rating widget increments, form validation	Manual (no framework-specific test runner mandated at this stage)
Integration testing	End-to-end flow: signup → login → rate → review → recommendations reflect the new rating	Spring Boot Test + Postman collections

Important test cases to include explicitly:

- Rating boundary tests: `1.0` and `5.0` accepted; `0.9` and `5.1` rejected; `4.3` (invalid increment) rejected; `4.5` accepted.
- Duplicate rating/update test: submitting a second rating for the same `(user, movie)` updates the existing row rather than creating a duplicate (verifies the unique constraint and upsert logic together).
- Unauthorized review modification test: User B attempting to edit/delete User A's review is rejected with `403 Forbidden`.
- Recommendation cold-start test: a user with zero ratings receives a non-empty response tagged `strategy: "cold-start"` rather than an error or empty list.

50. Development Workflow

Recommended incremental sequence:

1. Finalize project documentation
2. Review Figma design
3. Finalize database schema
4. Prepare movie data strategy
5. Create MySQL database
6. Import and validate movie metadata
7. Build frontend based on Figma
8. Create Spring Boot backend
9. Connect Spring Boot to MySQL

10. Implement movie APIs
11. Connect frontend movie cards to APIs
12. Implement authentication
13. Implement rating system
14. Implement review system
15. Implement content-based filtering
16. Implement TF-IDF
17. Implement cosine similarity
18. Implement item-based collaborative filtering
19. Implement hybrid recommendation system
20. Implement recommendation API
21. Connect recommendation frontend
22. Implement Cine Digest integration
23. Add trailer functionality
24. Test the full system
25. Optimize
26. Deploy

Why feature-by-feature: each step above produces a working, testable increment (e.g., the catalogue and cards work end-to-end before authentication exists, authentication works before ratings depend on it, ratings/reviews accumulate data before the recommendation engine needs it). This avoids a "big bang" integration where multiple untested subsystems are wired together simultaneously, and it matches the layered architecture described in Section 16.

51. Deployment Considerations

> **Decision Pending: hosting/deployment target.** Not yet finalized at the time of this document.
>
> - **Recommended (initial phase):** local development only (IntelliJ IDEA running Spring Boot, MySQL Workbench-managed local MySQL instance, static file serving for the frontend) until functional completeness is reached.

> - **Alternative:** containerize the backend (Docker) and deploy to a conventional VPS or managed Java hosting platform once the feature set stabilizes; frontend can be served as static files from any basic web host or the same server.
> - **Why recommended:** deployment decisions are premature before the core feature set (auth, ratings, reviews, recommendations, Cine Digest) is functionally complete and tested; revisit this section once development reaches that point.

General considerations to plan for when this is finalized: environment-specific configuration (DB credentials, TMDb API key, JWT secret) via environment variables or a Spring profile — never hardcoded into source control; CORS configuration updated for the production frontend origin; MySQL connection pooling settings appropriate for expected load.

52. AI Agent Development Rules

Any AI coding agent (Claude Code, Antigravity, Codex, or similar) working on CineSuggest must follow these rules:

1. Read this entire document before modifying CineSuggest.
2. Understand the existing codebase before creating new files.
3. Figma is the visual source of truth.
4. Do not redesign the frontend unless explicitly requested.
5. Frontend must remain HTML, CSS, and Vanilla JavaScript unless approval is given.
6. Do not migrate to React.
7. Backend must remain Java and Spring Boot.
8. Database must remain MySQL.
9. Use REST APIs and JSON.
10. Do not hardcode thousands of movie cards.
11. Movie cards must be dynamically generated.
12. Do not store plain-text passwords.
13. Do not store movie poster image binaries in MySQL.
14. Do not store trailer video files in MySQL.
15. Do not replace the finalized recommendation algorithms.

16. The finalized recommendation system includes Content-Based Filtering, TF-IDF, Cosine Similarity, Item-Based Collaborative Filtering, and Hybrid Recommendation.
17. Do not introduce deep learning, neural networks, TensorFlow, PyTorch, Random Forest, XGBoost, CNN, or LSTM unless explicitly requested.
18. Do not change the database schema silently.
19. Before major architectural changes, explain the proposed plan.
20. Preserve existing working functionality.
21. Avoid unnecessary dependencies.
22. Write maintainable and readable code.
23. Use clear naming.
24. Add comments only where they provide useful technical context.
25. Validate all user inputs.
26. Use DTOs where appropriate.
27. Keep controllers thin.
28. Keep business logic in services.
29. Keep database logic in repositories.
30. Keep recommendation logic modular.
31. Never expose external API secrets in frontend code.
32. Cine Digest and personalized recommendations must remain conceptually separate.
33. Ensure any generated code fits the existing project architecture.
34. Do not rewrite unrelated files.
35. When fixing bugs, identify the root cause before making broad changes.
36. Build features incrementally.
37. Explain which files are being created or modified before major implementations.
38. If project documentation conflicts with existing implementation, flag the conflict instead of silently making assumptions.
39. Never invent API response fields without checking the documented contract or existing backend.

40. Treat this master document as the primary project context unless a newer approved project document overrides it.

53. Current Mandatory Scope

The following constitute the **required** first implementation (restated from Section 4 for AI-agent clarity):

- Signup, login, logout
- Movie browsing, search, movie details
- Ratings (1.0–5.0, 0.5 increments, one active rating per user/movie, updatable)
- Reviews (create/read; edit/delete by the owning author)
- Trailer playback via external video reference
- Personalized recommendations: Content-Based Filtering + TF-IDF + Cosine Similarity + Item-Based Collaborative Filtering, combined via a Hybrid model
- Cold-start fallback strategy for new users
- Cine Digest (trending/popular/upcoming/new trailers), architecturally separate from recommendations
- MySQL persistence with the `users` / `movies` / `ratings` / `reviews` schema
- REST API layer via Spring Boot

Anything not listed here belongs in Future Enhancements and must not be silently folded into the current build.

54. Future Enhancements

Explicitly optional and NOT part of the required first implementation:

- Watchlist
- Favourite movies
- Social following
- Review likes
- Sentiment analysis / advanced NLP
- Actor/director discovery pages

- Notification system
- Admin moderation dashboard
- Advanced recommendation evaluation (A/B testing, offline metrics dashboards)
- React frontend migration
- Dedicated Python ML microservice
- Cloud deployment automation
- Redis caching

These must never be mixed into the current mandatory requirements without an explicit, documented decision to move them into scope.

55. Glossary

Term	Definition
TMDb	The Movie Database — external source of movie metadata (overview, genres, poster, release date, trailer key, popularity)
MovieLens	A research dataset of historical user-movie ratings, used to bootstrap collaborative filtering
TF-IDF	Term Frequency–Inverse Document Frequency — a technique for converting text into weighted numerical vectors that downweight common words
Cosine Similarity	A measure of similarity between two vectors based on the angle between them, used here to compare movie feature vectors
Content-Based Filtering	Recommending items similar to items a user already likes, based on the items' own features
Item-Based Collaborative Filtering	Recommending items based on rating-pattern similarity between items, derived from many users' behavior
Hybrid Recommendation	A combined score blending content-based and collaborative filtering signals
Cold Start	The problem of generating recommendations for a user (or item) with little/no historical data

JWT	JSON Web Token — a stateless token format used here for API authentication
DTO	Data Transfer Object — a class shaping API request/response payloads, decoupled from database entities
JPA / Hibernate	Java Persistence API and its common implementation, used to map Java entities to MySQL tables
CORS	Cross-Origin Resource Sharing — browser security mechanism that must be explicitly configured since frontend and backend run on different origins/ports in development
Cine Digest	CineSuggest's cinema trend/discovery module, explicitly separate from personalized recommendations
`links.csv`	The MovieLens file mapping `movieId` to external `tmdbId` / `imdbId`, used as the bridge for ID mapping

56. Project Status at Documentation Creation

As of the creation of this document, the current status of CineSuggest is:

- CineSuggest concept finalized
- Figma UI/UX design created
- Frontend coding not yet fully implemented
- MySQL and MySQL Workbench installed
- Cursor installed
- IntelliJ IDEA installed
- Antigravity and AI agents planned for frontend development
- Database design is being finalized
- Backend implementation has not yet started
- REST APIs have not yet been implemented
- Recommendation algorithms have been selected (Content-Based Filtering, TF-IDF, Cosine Similarity, Item-Based Collaborative Filtering, Hybrid) but not yet implemented
- Cine Digest has been added to project scope

- Dataset integration has not yet been completed

No feature described above as “planned,” “recommended,” or part of the mandatory scope should be treated as already built. This document describes the target architecture, not the current implementation state.

57. AI Context Quick Start

Before writing or modifying any CineSuggest code, an AI coding agent should:

1. **Read this entire document first** — it is the single source of truth for architecture, schema, API contracts, and the finalized recommendation algorithm set.
2. **Check Section 56** to understand what is and is not yet built — do not assume any feature exists until it’s confirmed in the actual codebase.
3. **Respect the fixed technology stack**: HTML/CSS/Vanilla JS frontend (Figma-driven), Java/Spring Boot backend, MySQL database, REST/JSON communication — do not substitute frameworks or languages.
4. **Respect the fixed recommendation algorithm set**: Content-Based Filtering, TF-IDF, Cosine Similarity, Item-Based Collaborative Filtering, Hybrid — do not substitute unrelated ML approaches.
5. **Treat any “Decision Pending” item** as unresolved — implement the recommended option only after confirming with the project owner, or clearly flag it as an assumption if asked to proceed unattended.
6. **Follow the AI Agent Development Rules** (Section 52) for every code change, however small.
7. **Build incrementally**, per the Development Workflow (Section 50), rather than attempting to implement multiple subsystems simultaneously.
8. **Flag, don’t silently resolve, any conflict** between this document and the actual existing implementation.